

Heuristics-based Symbolic Music Data Analysis Engine

Robert Cai¹

¹Faculty of Mathematics, University of Waterloo, Street, Canada

Abstract

A work in progress symbolic music analysis engine based off of heuristics (features) in the music score in order to generate a multitude of different musical annotations on a measure by measure basis. As opposed to the more popular Deep Neural Network approach, this engine aims to provide analysis of symbolic music based off of linear combinations of weights and heuristics alone, meaning that every decision is explainable. Currently, the engine is fitted, with different set of heuristics, with high accuracy, for the tasks of highlighting melodic parts in a polyphonic piece of symbolic music, as well as the task of highlighting parts that potentially deserves attention from the perspective of a conductor. We believe that there are many more improvements, heuristics as well as many more different musically informed uses this engine can be fitted with.

1. Introduction

Most Music Information Analysis tasks today involves the use of Machine Learning. In particular, the task of finding melody in a polyphonic (multi-part) music is primarily done through the use of Machine Learning, such as the MidiBERT tool from Chou et al. (2017). However, one major downside, despite the high accuracy of the tool, would be its lack of explainability.

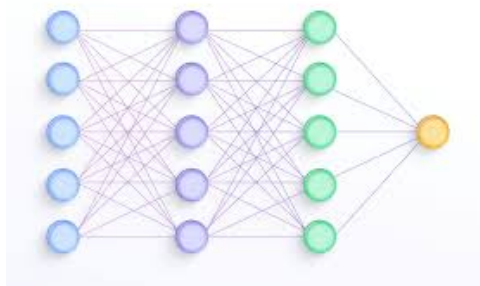


Figure 1: A Neural Network Structure. Note that "meaning" of each node is not explainable.

However, prior to the advent of Neural Networks, the predominant method of melody extraction relies on typically singular heuristics such as the Skyline method from Ozcan et al. (2005), where only the highest pitch is taken. A series of augmented versions of this algorithms are also detailed by Ozcan et al. However, the accuracy from this one heuristic alone is much to be desired.

As a result, we believe that an alternative approach using multiple explainable heuristics as criteria in such an expert system is a worthwhile approach. We model this approach after how a trained musician would identify melodies by inspection. We also note that this project uses



Figure 1. Illustration of our baselines. a) Input example, b) Output of skyline algorithm, c) Output of skyline variation algorithm.

Figure 2: The Skyline method and one of its augmentation.

some ideas from the AIST3110 course, detailed in later sections. We also use a similar approach 28
for the task of automated conductor annotation, but with a different set of heuristics. 29

2. Methodology

The tool uses primarily python in the form of Jupyter Notebook. Additionally, it primarily utilizes the music21 and mscore library for analysis.

2.1 Algorithm

We approach the score measure-by-measure. That is, we inspect each measure and extract feature values from each measure. We then take the parts with the highest score at each measure to be the parts that we keep as melody. This can be seen as follows.

```
k := number of instruments we keep at each measure
W := weights for each heuristic
S := score for each instrument at each measure
R := new score
for each measure in piece:
    for each instrument in measure:
        X := extract_feature() // A vector of feature values
        S.append(W @ S) // @ for dot product
    K := symbolic data of the top k instruments
    R.append(K)
return R
```

2.2 Feature Extraction

Different heuristics can be employed for different tasks. For the task of melody extraction, a series of the following heuristics are used.

- Dynamic (Loudness)
- Highest Pitches
- Significant Pitch Delta
- Short Rhythm
- Articulation Marks
- Continuity

We note that not all of the heuristics are taken as raw values. While the existence of Articulation marks is a simple 0-1 value, heuristics such as pitch delta needs to be normalized. There are two main ways of normalizing employed in the processing of the heuristics.

For example, for the pitch height, we simply either take all pitches higher than the mode as 1 or the k highest pitches as 1 and all the rest as 0.

$$f(pitch) = \begin{cases} 1 & \text{if } pitch > \text{mode or } pitch = \max \\ 0 & \text{otherwise} \end{cases}$$

Or, for the pitch delta, we simply take the maximum pitch delta in the measure as 1, and
 normalize (divide) the rest of the measures against the maximum to obtain a 0-1 value.

$$f(p\delta_i) = \frac{p\delta_i}{\operatorname{argmax}_{instruments}(p\delta)}$$

Similarly, for the annotation task, we use this set.

- Changing Time or Key Signature
- Difficult Ranges of Pitches
- Difficult Rhythm
- Accidentals

2.3 Training

We note that we need to obtain the weights that we multiply with the features. We use Multi-
 plicative Weight Update method, a method widely used in Machine Learning and Game Theory,
 to train our weights.

We update the weight of each wrong heuristic w_i at each iteration to be

$$w_i^{t+1} = w_i^t(1 - \lambda)$$

where λ is a fixed constant. We then normalize all the weights such that they sum to 1.

This gives us a guarantee in expectation polynomially bound by m^* minimum error and n
 number of heuristics (Aurora et al, 2012).

3. Results and Analysis

We first present the quantitative results for our tool, compared to the two different approaches in the Introduction section. We tested this algorithm on a section of Dvorak's New World Symphony Movement IV. The accuracy score for our method is filled in from this test.

Heuristic	Skyline	CNN
88.9%	78.7%	91.7%

We can clearly see that, while the result is not as ideal as the Machine Learning model, it is still significantly improved compared to the Skyline model.

On further analysis of the results, we take a look into some of the "mistakes" that the Heuristics model made.

Figure 3: The mistake at bar 7-8 on Test Piece Liz. Left is original score while right is the result after analysis.

For the mistake in Figure 3, we observe that at bar 7-8, the melody should be on the first and third part (flute and oboe), due to the continuity of the expression from earlier as it observed. However, due to the harp (the part at the bottom right) having a shorter rhythm it find the harp more appealing.

For the mistake in Figure 4, we observe that while the trumpet should retain the melody, as the trumpet is inherently louder than the violin, it is not reflected on the sheet as both parts are *ff* and thus the analysis favours the shorter rhythm in the violin, committing to a mistake.

After some tests we collected a set of possible reasons and thus improvements on the model.

- Missing Pickups (which we now mark for manual inspection at every bar, thus partially resolving the issue).
- Long Ending Notes (Figure 3.)
- Counter-melody



Figure 4: The mistake at bar 11 on the New World Symphony. Left is original score while right is the result after analysis.

- Absolute Volume (Figure 4.)

100

We recognize that it is often difficult to distinguish between counter-melody and main melody, as such, the only possible improvement we can offer is sectional analysis, potentially using tools such as Self-Similarity Matrix on expressions rather than measures, to possibly distinguish the importance of each expression.

101

102

103

104

Similar to the Melody Detection improvements, we also find some potential improvements with the Annotation tool. In particular, we find that the lack of more sophisticated (or more fine-grained) analysis hampers the results. For example, one aspect that could be improved is chord analysis across different parts; however, this would occasionally require analysis on a note-by-note basis as opposed to a measure-by-measure approach, since multiple chords could persist in each measure, but this brings much more difficulty with regards to the iteration of the tool.

105

106

107

108

109

110

Additionally, more sophisticated traits are often required, such as identifying duets in a piece, but these may require multiple different simple heuristics to identify the feature, thus making the annotation more difficult.

111

112

113

For the annotation task, we propose that more different heuristics to be used to further identify interesting parts.

114

115

4. References	116
G. Ozcan, C. Isikhan and A. Alpkocak, "Melody extraction on MIDI music files," Seventh IEEE International Symposium on Multimedia (ISM'05), Irvine, CA, USA, 2005, pp. 8 pp.-, doi: 10.1109/ISM.2005.77.	117 118 119
K. Katerina, L. Wei Tsung, M. Gabriele, C. Pierre, "A deep learning method for melody extraction from a polyphonic symbolic music representation", ISMIR 2012, 2012.	120 121 122
S. Arora, E. Hazan, S. Kale, "The Multiplicative Weights Update Method: a Meta-Algorithm and Applications." Theory of Computing, 2012.	123 124 125
Y. Chou, I. Chen, C. Chang, J. Ching, Y. Yang, MidiBERT-Piano: Large-scale Pre-training for Symbolic Music Understanding, arXiv, 2021, 2107.05223	126 127 128 129